Workshop n°1 2026-2027	Module	MTI Methods and techno. of Impl.
	Branch	Master GSI 1st year
Chapter 1: Typo. lang. Prog.	ملاحظة :	

0.1 Part 1

Objectives : Learn Python language

Toolsأدوات : Python, PyScripter/PyCharm / VScode

Work to do للعمل

- Install Python under Windows or Linux تثبيت بيثون على وندوز أو لينكس
- Run a Python script تنفيذ برنامج بلغة بيثون
- Write programs with Python كتابة برامج بلغة بيثون

Run the following code نفّذ ما يلي

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
# character encoding declaration, often used UTF-8,
# to support all multilingual.
print("Salam Alykom")

#  Variables declaration
# no explicit declaration
# just assign a value to specify the type of a variable
age = 125 # integer
print ("type of age", type(age))
salaire = 3.14
print ("type of salaire", type(salaire))
name = "Ahmed"
print ("type of name", type(name))

#substitution
s2 = "I am %s"%name
s3 = "I am %d old "%age
print(s2, s3)


# listes
liste = [1,2,31,4]
print ("type of ls", type(liste))
print (liste)
# add an element
liste.append(5)
print (liste)

# word list

wordlist = ["I", "am", "learning", "english"]
```

```python
# Flows Control
a = 15
if a > 15:
    # pay attention to tabs
    print ('a is greater than 15')
elif a <= 15 and a >= 10:
    print ('a between  10 and 15')
else:
    print ('a is less than 10')



# test if an element belongs to a list
a = 5
if a in liste:
    print ("a exists in list")


# Loops
for word in wordlist:
    print (word)

# Loops and tests
# Count occurrences of a character in a string
chaine = """The sky, oh the sky! It stretches above us, vast and endless.
The sky is a canvas, a canvas of blue, blue like the ocean on a clear summer day.
 It's a canvas of possibilities, possibilities as vast as the sky itself."""
caractere="e"
cpt = 0
for c in chaine:
    if c == caractere:
        cpt += 1
print ("Char %s exists %d times"%(caractere, cpt))
```

Run the following code: be careful the code contains errors.

نفّذ البرنامج الآتي، انتبه فيه بعض الأخطاء.

```
def clean_text(text):
     "
        remove punctuation  form a text
     '
     punct = '+.*/?,;\'#
       for  p  in  punct
              text  =  text.replace(p,")
      return   text

def  tokenize(text)
    "
       Convert  text into tokens,  return  a list of  tokens(words)
       "''
      return  text.split( )
def tokenize2(text)
      '
      Convert  text  into tokens,  return a list  of tokens(words)
       "'
```

```
      spaces = r\s+
    words = re.split(spaces,  text)
        return word


def  word_freq(words):

      Counts  words and return a  dictionary of words  with  their occurrences

       index  = {}
    for w in  words:
             if w in  index
              index[w]  += 1
           else
                index[w] =  1
      return index
def most_common_word(words_freq_table)
       '
    Counts words  and return a  dictionary of  words with their  occurrences
      ''
      frequent =
     frequency  = 0
       for  word in  words_freq_table
            if  words_freq_table[word] >  frequency:
              frequent  =  word
                 frequency =  words_freq_table[word]
      return frequent




def read_file(filename)
     ''  Read  a  text from  file

      text =  '''
      try
            fl =  open(filename)
      except:
              print(Can't open  file,  ",  filename)
            sys.exit()
     #  if  success
       text  = fl.read()
      lines = fl.readlines()
     return text

def  main(args):
    text =  ""  Surprise steepest recurred landlord mr  wandered amounted  of.
     Continuing  devonshire but  considered its.  Rose  past oh shew  roof
    is song neat. Do depend  better praise do friend garden  an wonder  to.
        Intention age nay otherwise  but  breakfast.
        Around garden beyond  to  extent by.
      "
     text =  clean_text(text)
       print(text)
     #  tokenize  text
      words  = tokenize(text)
     print(words)
       words_nb = word_freq(words)
```

```
      print(words_nb)
    data  = read_file("data.txt)
      print(data)
      text  =  data
     words  =  tokenize(text)
      print(words)
      word_freq_table = word_freq(words)
     print(word_freq_table)
      freqw  =  most_common_word(word_freq_table)
       print(most frequent word is  ',  freqw,  word_freq_table[freqw]  )
      return  0
```

0.2 Part 2

Objectives : Read files and use dictionaries and lists

Toolsأدوات : Python

Here is a data CSV file representing M1 GSI canvas:

فيما يلي ملف بيانات CSV يمثل لوحة GSI: M1

```
semestre;unite;unite_title;module_name;module_title;coef;credit;percent_tp;percent_td;percent_exam
S1;U1_S1;Fondamentale1;RCB;Réseaux des couches basses;3;6;20;20;60
S1;U1_S1;Fondamentale1;AlgoAv;Algorithmique Avancée et Complexité;2;4;40;0;60
S1;U2_S1;Fondamentale2;SE;Système d'exploitation;2;4;0;40;60
S1;U2_S1;Fondamentale2;AMSI;Architectures Modernes des Systèmes Informatiques;2;4;40;0;60
S1;U3_S1;Méthodologie;AABD;Architecture et administration des bases de données;2;4;40;0;60
S1;U3_S1;Méthodologie;MTI;Méthodes et Technologies d'Implémentation;3;5;40;0;60
S1;U4_S1;Découverte;scvv;Systèmes de Communication  Vocaux et Vidéos;2;2;40;0;60
S1;U5_S1;Transversale;cloud;Cloud Computing;1;1;0;0;100
S2;U1_S2;Fondamentale1;Middleware;Les Middlewares pour les systèmes répartis;3;6;20;20;60
S2;U1_S2;Fondamentale1;MAL;Modélisation et Architectures logicielles;2;4;40;0;60
S2;U2_S2;Fondamentale2;ResIP;Réseaux de la couche IP;3;6;40;0;60
S2;U2_S2;Fondamentale2;ML;Introduction au Machine Learning;2;4;40;0;60
S2;U3_S2;Méthodologique;Infog;Infographie;2;3;40;0;60
S2;U3_S2;Méthodologique;GI;Gestion de l'incertain;2;3;0;40;60
S2;U4_S2;Découverte;IoT;Internet of Things;2;3;40;0;60
S2;U4_S2;Transversale;Sec;Cybercriminalité;1;1;0;0;100
```

Work to do للعمل

- Use csv library to read this file استعمل مكتبة csv لقراءة هذا الملف
- Run a Python script to convert data into dictionary with two levels of keys (semester then unit) تشغيل برنامج بيثون لتحويل البيانات إلى قاموس بمستويين للمفاتيح (السداسي ثم الوحدة)
- Modify the code to add a new level for keys (semester, unit then module) عدّل البرنامج لإضافة مستوى آخر للمفاتيح (السداسي ثم الوحدة ثم المادة)
- Display result as Json اعرض النتيجة بصيغة JSON.

```python
import csv
from collections import defaultdict

def csv_to_nested_dict(csv_file):
    # nested dictionary format
    data = {"S1":{},
            "S2":{}}

    with open(csv_file, encoding='utf-8') as f:
        reader = csv.DictReader(f, delimiter="\t")
```

```
        for row in reader:
            semester = row['semester']
            unite = row['unit']

            # build module as dictionarryary
            module = {
                "module_name": row['module_name'],
                "module_title": row['module_title'],
                "coef": int(row['coef']),
                "credit": int(row['credit']),
                "percent_tp": float(row['percent_tp']),
                "percent_td": float(row['percent_td']),
                "percent_exam": float(row['percent_exam'])
            }
            # init a unite as list
            if not unite in data[semester]:
                 data[semester][unite] = []
            data[semester][unite].append(module)

    return data

# --------- Test ---------
if __name__ == "__main__":
    filename = "modules.csv"   # change to your file name
    result = csv_to_nested_dict(filename)

    print(result)
```

Convert Data into HTML format: تحويل البيانات إلى صيغة HTML

```


def modules_to_html(data):
    html = ['<html><head><meta charset="utf-8">',
            '</head><body>']

    for semester, unites in data.items():
        html.append(f"<h2> Semester {semester}</h2>")
        for unite, modules in unites.items():
            html.append(f"<h3>{unite}</h3>")
            html.append("<table>")
            html.append("<tr><th>Module Code</th><th>Title</th><th>Coef</th><th>Credit</th>"
                        "<th>TP%</th><th>TD%</th><th>Exam%</th></tr>")
            for info in modules:
                html.append("<tr>")
                html.append(f"<td>{info['module_name']}</td>")
                html.append(f"<td>{info['module_title']}</td>")
                html.append(f"<td>{info['coef']}</td>")
                html.append(f"<td>{info['credit']}</td>")
                html.append(f"<td>{info['percent_tp'] * 100:.0f}%</td>")
                html.append(f"<td>{info['percent_td'] * 100:.0f}%</td>")
                html.append(f"<td>{info['percent_exam'] * 100:.0f}%</td>")
                html.append("</tr>")
            html.append("</table>")
        html.append("<hr>")

    html.append("</body></html>")
```

```
    return "\n".join(html)

# --------- Test ---------
if __name__ == "__main__":
    filename = "modules.csv"    # change to your file name
    result = csv_to_nested_dict(filename)

    print(result)
    html_content = modules_to_html(result)

    with open("modules.html", "w", encoding="utf-8") as f:
        f.write(html_content)

    print("HTML file generated: modules.html")
```

0.3 Home work الواجب المنزلي

Realize the following tasks for the given project.

- Read the Home work recommendations.
- Read the project description.
- Do only the following tasks:
  - Learn the **Python** fundamentals required to complete the project.
  - Create a game prototype using **Pygame**.
  - Create a **player** who moves within the game area.
  - Create a **monster** that moves within the game.
  - Set up the main game loop, input handling, object updates, and rendering.
  - **AI tools may be used** during development, provided that students understand the code used and are able to explain and modify it.

أنجز المهام التالية المتعلقة بالمشروع المقترح:

- قراءة توصيات إعداد تقرير للواجب المنزلي.
- قراءة وصف المشروع.
- تنفيذ المهام التالية فقط:
  - تعلّم أساسيات لغة **Python** اللازمة لإنجاز المشروع.
  - إنشاء نموذج أولي للعبة باستخدام مكتبة **Pygame**.
  - إنشاء **بطل** يتحرك داخل مساحة اللعب.
  - إنشاء **وحش** يتحرك داخل اللعبة.
  - إعداد الحلقة الرئيسية للعبة ومعالجة الإدخال وتحديث الكائنات وعرضها.
  - **يمكن الاستعانة بأدوات الذكاء الاصطناعي** أثناء الإنجاز، مع ضرورة فهم الشيفرة المستخدمة والقدرة على شرحها وتعديلها.

1 Project: 2D Shooting Game

1.1 General Idea

Develop a 2D shooting game set in a world of the student's choice. The player controls a main character who confronts enemies and attempts to defeat them using the available weapons and abilities, while collecting rewards and overcoming various challenges.

1.1.1 Game Elements

- **Player:** Moves within the game and shoots at the monster.
- **Monster:** Attacks the player and attempts to defeat them.
- **Bullet:** Fired by the player to hit and eliminate the monster.
- **Reward:** Obtained by the player after defeating the monster, providing points or an additional benefit.

The game progressively evolves by adding new characters, enemies, weapons, rewards, and levels, eventually resulting in a complete and extensible game.

Students may choose the game's story, characters, and world, while maintaining the core elements and requirements of the project.

1.2 Key Features to Consider

- Player movement and shooting.
- Monster pursuit and attacks.
- Bullet–monster collisions.
- Defeating monsters and collecting rewards.
- Health and score management.
- Different types of enemies and rewards.
- Gradually increasing difficulty.
- Multiple levels and challenges.

1.3 Technical Stack Suggestions

- **Python:** The main programming language for developing the project.
- **Pygame or Arcade:** For graphics, movement, input handling, collision detection, and sound.
- **Git / GitHub:** For version control and managing the development process.
- **OOP:** For representing game elements and organizing their interactions.
- **SOLID:** For improving the design and making it maintainable and extensible.
- **MVC:** For separating game logic, presentation, and input handling.
- **Design Patterns:** For solving advanced design problems by selecting appropriate patterns.
- **Markdown or LaTeX:** For project documentation and the final report.

1.4 Workflow

- **Object creation:** Creating the player, monsters, bullets, and rewards.
- **Interaction:** The player moves and shoots, while monsters move and attack the player.
- **Collision:** Checking whether bullets hit monsters or monsters hit the player.
- **State changes:** Defeating a monster, losing health, or obtaining a reward.
- **Update and rendering:** Updating object states and then rendering them in the game.

1.5 Data Fields

- **Player:** Position, speed, health, weapon, score.
- **Monster:** Position, speed, health, attack type.
- **Bullet:** Position, speed, direction, damage.
- **Reward:** Position, type, value.

1.6 Variation

Students may choose the **game story, characters, monster types, weapons, and rewards**, while respecting the core elements and requirements of the project. Additional features or similar ideas may be proposed with the instructor's approval, so that **the objectives remain common while the implementations may vary**.

Non-exhaustive examples:

- **Space:** An astronaut defends their spacecraft against attacking aliens.
- **City:** A fighter attempts to protect the city from invading monsters.
- **Forest:** A hunter confronts strange creatures that have appeared in the forest.
- **Future:** A robotic soldier fights against an army of rebellious robots.
- **Zombies:** A survivor attempts to reach a safe area while facing waves of zombies.
- **Free Story:** The student creates their own world and story while preserving the project's core elements.

1.7 Work to Do

The practical work consists of developing a set of **game features and functionalities**, starting with basic functions and progressing toward more advanced ones, with an emphasis on the practical application of programming concepts while developing these features. Examples include:

- **Player movement:** Movement, direction changes, and movement boundaries.
- **Attack:** Shooting bullets and determining their direction, speed, and effect.
- **Enemies:** Movement, pursuit, attacks, and diverse behaviors.
- **Collision:** Detecting monsters hitting the player or bullets hitting monsters.
- **Health and score:** Losing health, defeating enemies, and calculating scores.
- **Rewards:** Generating and collecting rewards and applying their effects to the player or score.
- **Weapons:** Adding multiple weapons with different properties and behaviors.
- **Game levels:** Different stages, challenges, and difficulty levels.

- **Game interface:** Displaying score and health, menus, and different game states.
- **Extensibility:** Adding new elements and features while maintaining well-organized and maintainable code.

In this way, the project:

- Starts **simple** (a game with a player and a monster).
- Progressively evolves to cover **all the concepts of the course** (OOP, SOLID, MVC, Design Patterns, AOP, Frameworks).
- Is developed through weekly stages linked to the theoretical lessons.

Note to Students:
You are required to use **GitHub** throughout this project. Each team member must contribute actively by:

- Working collaboratively through GitHub repositories.
- Committing and pushing changes progressively (**not all at once at the end**).
- Using branches or pull requests when appropriate.
- Documenting progress in the repository (README, commit messages, issues, etc.).

This will help you learn collaborative development, version control, and professional software development practices.

2 مشروع: لعبة إطلاق نار

2.1 الفكرة العامة

تطوير لعبة إطلاق نار ثنائية الأبعاد تدور أحداثها في عالم يختاره الطالب. يتحكم اللاعب في شخصية رئيسية تسعى إلى مواجهة الأعداء والتغلب عليهم، باستخدام الأسلحة والقدرات المتاحة، وجمع المكافآت وتجاوز مختلف التحديات.

2.2 عناصر اللعبة

- **اللاعب :(Player)** يتحرك داخل اللعبة ويطلق النار على الوحش.
- **الوحش :(Monster)** يهاجم اللاعب ويحاول القضاء عليه.
- **الطلقة :(Bullet)** يطلقها اللاعب لإصابة الوحش والقضاء عليه.
- **المكافأة :(Reward)** يحصل عليها اللاعب بعد القضاء على الوحش، وتمنحه نقاطًا أو فائدة إضافية.

تتطور اللعبة تدريجيًا بإضافة شخصيات وأعداء وأسلحة ومكافآت ومراحل جديدة، وصولًا إلى لعبة متكاملة وقابلة للتوسعة.
يمكن للطالب اختيار قصة اللعبة وشخصياتها وعالمها، مع الحفاظ على العناصر الأساسية المطلوبة في المشروع.

2.3 المزايا الرئيسية

- حركة اللاعب وإطلاق النار.
- مطاردة الوحش ومهاجمة اللاعب.
- اصطدام الرصاص بالوحش.
- القضاء على الوحش والحصول على مكافأة.
- احتساب النقاط والحياة.
- ظهور أعداء ومكافآت متنوعة.
- تطور مستوى الصعوبة تدريجيًا.
- تعدد المراحل والتحديات.

2.4 البنية التقنية

- **Python:** لغة البرمجة الأساسية لتطوير المشروع.
- **Pygame أو Arcade:** تطوير الرسوميات، الحركة، الإدخال، التصادمات والصوت.
- **GitHub: / Git** إدارة الإصدارات وتنظيم تطور المشروع.
- **OOP:** تمثيل عناصر اللعبة وتنظيم تفاعلها.
- **SOLID:** تحسين التصميم وجعله قابلًا للصيانة والتوسعة.
- **MVC:** فصل منطق اللعبة عن العرض ومعالجة الإدخال.
- **Patterns: Design** معالجة مشكلات التصميم المتقدمة باختيار الأنماط المناسبة.
- **Markdown أو LaTeX:** توثيق المشروع وإعداد التقرير النهائي.

2.5 سير العمل

- **إنشاء الكائنات:** إنشاء اللاعب، الوحوش، الرصاص والمكافآت.
- **التفاعل:** يتحرك اللاعب ويطلق الرصاص، وتتحرك الوحوش وتهاجم اللاعب.
- **التصادم:** التحقق من اصطدام الرصاص بالوحش أو الوحش باللاعب.
- **تغيير الحالة:** القضاء على الوحش، فقدان الحياة، أو الحصول على مكافأة.
- **التحديث والعرض:** تحديث حالات الكائنات ثم عرضها في اللعبة.

2.6 حقول البيانات

- **Player:** الموقع، السرعة، الصحة، السلاح، النقاط.
- **Monster:** الموقع، السرعة، الصحة، نوع الهجوم.
- **Bullet:** الموقع، السرعة، الاتجاه، الضرر.
- **Reward:** الموقع، النوع، القيمة.

2.7 التنويع

يمكن للطالب اختيار **قصة اللعبة، الشخصيات، أنواع الوحوش، الأسلحة والمكافآت**، مع الالتزام بالعناصر والمتطلبات الأساسية للمشروع. ويُسمح باقتراح إضافات أو أفكار مشابهة بعد موافقة الأستاذ، بحيث تكون **الأهداف موحّدة والتنفيذ متنوعًا.**
أمثلة غير حصرية:

- **الفضاء:** رائد فضاء يدافع عن مركبته ضد كائنات فضائية مهاجمة.
- **المدينة:** مقاتل يحاول حماية المدينة من الوحوش التي اجتاحتها.
- **الغابة:** صياد يواجه مخلوقات غريبة ظهرت في الغابة.
- **المستقبل:** جندي آلي يقاوم جيشًا من الروبوتات المتمردة.
- **الزومبي:** ناجٍ يحاول الوصول إلى منطقة آمنة وسط موجات من الزومبي.
- **القصة الحرة:** يبتكر الطالب عالمًا وقصة خاصة به، بشرط المحافظة على العناصر الأساسية للمشروع.

2.8 أعمال تطبيقية

تتناول الأعمال التطبيقية تطوير مجموعة من **مزايا اللعبة ووظائفها**، بدءًا من الوظائف الأساسية وصولًا إلى الوظائف الأكثر تقدمًا، مع التركيز على التطبيق العملي للمفاهيم البرمجية أثناء تطوير هذه المزايا. وتشمل المهام، على سبيل المثال:

- **حركة اللاعب:** التنقل، تغيير الاتجاه، والحدود المسموح بها.
- **الهجوم:** إطلاق الرصاص، تحديد اتجاهه وسرعته وتأثيره.
- **الأعداء:** الحركة، المطاردة، الهجوم، وتنوع السلوكيات.
- **التصادم:** التحقق من إصابة الوحوش للاعب أو إصابة الرصاص للوحوش.
- **الحياة والنقاط:** فقدان الصحة، القضاء على الأعداء، واحتساب النقاط.

- **المكافآت:** ظهور المكافآت وجمعها وتأثيرها على اللاعب أو النقاط.
- **الأسلحة:** إضافة أسلحة متعددة تختلف في الخصائص والسلوك.
- **مستويات اللعبة:** مراحل وتحديات ودرجات مختلفة من الصعوبة.
- **واجهة اللعبة:** عرض النقاط والصحة، والقوائم، وحالات اللعب المختلفة.
- **التوسعة:** إضافة عناصر ومزايا جديدة مع المحافظة على تنظيم الشيفرة وقابليتها للتطوير.

 بهذه الطريقة نضمن أن المشروع:

- يبدأ **بسيطًا** (لعبة بطل ووحش).
- ثم يتطور تدريجيًا ليغطي **كل مفاهيم المادة** (OOP, AOP, patterns, Design MVC, SOLID, Frameworks).
- تُنجز على مراحل أسبوعية مرتبطة بالدروس النظرية.

ملاحظة للطلبة: يُطلَب منكم استعمال **GitHub** في هذا المشروع، بحيث يقوم كل فرد من الفريق بالمساهمة بشكل فعّال من خلال:

- العمل الجماعي عبر مستودعات GitHub.
- رفع التعديلات تدريجيًا (**وليس دفعة واحدة في النهاية**).
- استخدام الفروع (branches) أو طلبات السحب (pull requests) عند الحاجة.
- توثيق التقدم في المستودع (ملف README، رسائل الـ commit، القضايا/issues، إلخ).

هذا سيساعدكم على تعلّم العمل الجماعي، وإدارة النسخ، والممارسات الاحترافية في التطوير البرمجي.

3 How to write a report

Note : The recommendations for preparing a report must be followed.

3.0.1 Title page

You must specify the following information on the cover page:

- Binomial names
- Specialty and Group
- Module
- Practical work number + Project Title.

3.0.2 Report Structure :

1. **Introduction** : explaining the problem to be solved and the questions asked
2. **Methodology** : the technique used (methods and algorithms)
3. **Tools** : to cite the libraries, frameworks, tools to realize the work.
   Avoid representing tools that are presented in lesson or already in use.
4. **Tests and results**: present examples of input data and results after execution.
5. **Insert code in appendices** : structure the code and provide it with comments explaining the important things.

3.0.3 Remark :

- No more than two people (a pair)
- Use only clamps (without bookbinder).
- Avoid plagiarism (all similar works are rejected and rated to Zero)
- If you want to help your colleague, help him learn.
- Colors are not necessary.

3.0.4 Submission :

- The report must be submitted within the scheduled dates.
- Submission of the report online under the platform.
- The delay in submitting the report is taken into account.
  To submit:
  - The report of the work done during the practical session.
  - Home working.

GitHub recommendations: You are required to use **GitHub** for this project. Each team member should actively contribute by:

- Collaborating through GitHub repositories.
- Committing and pushing your work **gradually** (not all at once at the end).
- Using branches or pull requests when needed.
- Documenting your progress in the repository (README, commit messages, issues, etc.).

3.1 كيفية إعداد تقرير

ملحوظة : يجب اتباع توصيات إعداد تقرير العمل التطبيقي.

3.1.1 صفحة الغلاف

في **صفحة الغلاف** ، يجب عليك تحديد المعلومات التالية:

- أسماء أعضاء الفريق
- المستوى التخصص / الفوج
- اسم المادة
- رقم العمل التطبيقي و عنوان المشروع.

3.1.2 هيكل التقرير:

1. **مقدمة:** شرح المسألة والمطلوب
2. **منهجية العمل:** التقنية المستخدمة (الطرائق والخوارزميات)
3. **الأدوات:** اذكر المكتبات والأدوات وأطر العمل اللازمة لإنجاز المطلوب.
   تجنب ذكر الأدوات التي سبق تقديمها في الدرس أو سبق استخدامها.
4. **الاختبارات والنتائج**: تقديم أمثلة لبيانات الإدخال والنتائج بعد التنفيذ.
5. **متن البرنامج المصدري في الملحق**: صُغ البرنامج صياغة جيدة، وزوّده بتعليقات تشرح الأشياء المهمة.

3.1.3 ملاحظات

- لا يزيد الفريق عن عضوين اثنين
- أمسك الورق بالدبّاسة (لا داعي لاستعمال التغليف).
- تجنب الغش والانتحال (تُرفض جميع الأعمال المتماثلة وتُصفّر)
- إذا كنت ترغب في مساعدة زميلك ، ساعده على التعلم.
- الطباعة الملونة ليست ضرورية.

3.1.4 تسليم

- يجب تسليم التقرير في المواعيد المحددة.
- تسليم الواجب في منصة التعليم الالكتروني
- يُحتسب التأخر في التسليم
- يجب تسليم :
  - تقرير الأعمال المنفذة خلال حصة الأعمال التطبيقية.
  - الواجب المنزلي.

استعمال :Github يُطلَب منكم استعمال **GitHub** في هذا المشروع، بحيث يساهم عضو الفريق مساهمة فعّالة بـ:

- العمل الجماعي عبر مستودعات GitHub.
- رفع التعديلات تدريجيًا (**وليس دفعة واحدة في النهاية**).

- استخدام الفروع (branches) أو طلبات الدمج (pull requests) عند الحاجة.
- توثيق التقدم في المستودع (ملف ،README رسائل الـ commit، المشاكل/ issues، إلخ).

هذا سيساعدكم على تعلّم العمل الجماعي، وإدارة النسخ، والممارسات الاحترافية في التطوير البرمجي.